

Quantum-Correction

Source-code guide and project architecture

Purpose

This guide explains the current surface-code quantum error-correction simulation. The project first establishes a trusted IID-noise baseline, then provides the foundation for the planned Ornstein-Uhlenbeck temporal-noise and correlation-aware-decoding research.

Current pipeline	Surface-code circuit -> IID noise -> Stim sampling -> detector events -> MWPM decoding -> logical error rate
-------------------------	--

1. Research objective

The project asks how temporal correlations in physical errors affect surface-code correction, and whether a decoder that understands temporal correlations can outperform standard MWPM. The intended comparison is between the noise correlation time τ_c and decoder memory T , with the hypothesis that decoder improvements become visible when T is comparable to or larger than τ_c .

The IID baseline is the control experiment: errors have no time memory. Its logical-error curves will later be compared with curves produced by correlated OU noise under the same code distances, rounds, and sampling procedure.

2. How the files connect

Layer	File	Responsibility
Project documentation	README.MD	See the detailed file section for the complete code and connection to the pipeline.
Python dependencies	requirements.txt	See the detailed file section for the complete code and connection to the pipeline.
Git exclusions	.gitignore	See the detailed file section for the complete code and connection to the pipeline.
Package marker	src/__init__.py	See the detailed file section for the complete code and connection to the pipeline.
Circuits package marker	src/circuits/__init__.py	See the detailed file section for the complete code and connection to the pipeline.
Surface-code circuit factory	src/circuits/surface_code.py	See the detailed file section for the complete code and connection to the pipeline.
Noise package marker	src/noise/__init__.py	See the detailed file section for the complete code and connection to the pipeline.
IID noise model	src/noise/iid_noise.py	See the detailed file section for the complete code and connection to the pipeline.
Decoder package marker	src/decoders/__init__.py	See the detailed file section for the complete code and connection to the pipeline.
MWPM decoder factory	src/decoders/mwpm.py	See the detailed file section for the complete code and connection to the pipeline.
Baseline experiment	experiments/baseline_mwpm.py	See the detailed file section for the complete code and connection to the pipeline.
Original setup test	tests/test_mwpm_setup.py	See the detailed file section for the complete code and connection to the pipeline.
IID baseline tests	tests/test_iid_baseline.py	See the detailed file section for the complete code and connection to the pipeline.

3. README.MD

Project documentation. Complete source code and explanation of its role in the project.

Complete file contents

(empty file)

How it connects: Supports package organization, dependency installation, documentation, or repository hygiene.

4. requirements.txt

Python dependencies. Complete source code and explanation of its role in the project.

Complete file contents

```
stim==1.13.0  
pymatching==2.2.0  
numpy==1.26.4  
matplotlib  
pytest
```

How it connects: Supports package organization, dependency installation, documentation, or repository hygiene.

5. .gitignore

Git exclusions. Complete source code and explanation of its role in the project.

Complete file contents

```
results/*.csv  
results/*.png  
__pycache__/  
*.pyc  
.DS_Store
```

How it connects: Supports package organization, dependency installation, documentation, or repository hygiene.

6. src/___init___.py

Package marker. Complete source code and explanation of its role in the project.

Complete file contents

(empty file)

How it connects: Supports package organization, dependency installation, documentation, or repository hygiene.

7. src/circuits/__init__.py

Circuits package marker. Complete source code and explanation of its role in the project.

[Complete file contents](#)

(empty file)

How it connects: Supports package organization, dependency installation, documentation, or repository hygiene.

8. src/circuits/surface_code.py

Surface-code circuit factory. Complete source code and explanation of its role in the project.

Complete file contents

```
import stim

from src.noise.iid_noise import get_iid_noise


def create_surface_code(distance: int, rounds: int, error_rate: float = 0.0):
    """
    Create a rotated surface-code memory circuit using Stim.
    """

    if distance < 3 or distance % 2 == 0:
        raise ValueError("distance must be an odd integer >= 3")
    if rounds < 1:
        raise ValueError("rounds must be a positive integer")

    noise = get_iid_noise(error_rate)

    circuit = stim.Circuit.generated(
        "surface_code:rotated_memory_z",
        distance=distance,
        rounds=rounds,
        **noise,
    )

    return circuit
```

How it connects: Creates the Stim rotated Z-memory circuit and applies noise settings. Its returned circuit is consumed by the sampler and the MWPM decoder.

9. src/noise/___init___.py

Noise package marker. Complete source code and explanation of its role in the project.

[Complete file contents](#)

(empty file)

How it connects: Supports package organization, dependency installation, documentation, or repository hygiene.

10. src/noise/iid_noise.py

IID noise model. Complete source code and explanation of its role in the project.

Complete file contents

```
def get_iid_noise(error_rate: float):
    """
    Create independent (IID) noise settings.

    Parameters
    -----
    error_rate : float
        Probability of a physical error.

    Returns
    -----
    dict
        Noise parameters that can be passed to Stim.
    """
    if not 0 <= error_rate <= 1:
        raise ValueError("error_rate must be between 0 and 1")

    noise = {
        "after_clifford_depolarization": error_rate,
        "before_measure_flip_probability": error_rate,
        "after_reset_flip_probability": error_rate,
    }

    return noise
```

How it connects: Defines the independent physical-noise parameters. It keeps noise modeling separate so OU noise can later replace it without rewriting the circuit or experiment layers.

11. src/decoders/__init__.py

Decoder package marker. Complete source code and explanation of its role in the project.

[Complete file contents](#)

(empty file)

How it connects: Supports package organization, dependency installation, documentation, or repository hygiene.

12. src/decoders/mwpm.py

MWPM decoder factory. Complete source code and explanation of its role in the project.

Complete file contents

```
import pymatching

def create_mwpm_decoder(circuit):
    """
    Create an MWPM decoder from a Stim surface-code circuit.
    """
    detector_error_model = circuit.detector_error_model(
        decompose_errors=True
    )
    matching = pymatching.Matching.from_detector_error_model(
        detector_error_model
    )
    return matching
```

How it connects: Builds the PyMatching decoder from Stim detector data. It receives detector events and predicts logical observables.

13. experiments/baseline_mwpm.py

Baseline experiment. Complete source code and explanation of its role in the project.

Complete file contents

```
import argparse
import csv
from pathlib import Path

import numpy as np

from src.circuits.surface_code import create_surface_code
from src.decoders.mwpm import create_mwpm_decoder

DEFAULT_DISTANCES = (3, 5, 7)
DEFAULT_ERROR_RATES = (0.0001, 0.0003, 0.001, 0.003, 0.01)

def run_point(distance, error_rate, shots):
    circuit = create_surface_code(distance, distance, error_rate)
    decoder = create_mwpm_decoder(circuit)
    sampler = circuit.compile_detector_sampler()
    detectors, observables = sampler.sample(
        shots=shots, separate_observables=True
    )
    predictions = decoder.decode_batch(detectors)
    return int(np.any(predictions != observables, axis=1).sum())

def run_baseline(distances=DEFAULT_DISTANCES, error_rates=DEFAULT_ERROR_RATES, shots=10_000):
    if shots < 1:
        raise ValueError("shots must be positive")
    results = []
    for distance in distances:
        for error_rate in error_rates:
            failures = run_point(distance, error_rate, shots)
            row = {
                "distance": distance,
                "error_rate": error_rate,
                "rounds": distance,
                "shots": shots,
                "logical_failures": failures,
                "logical_error_rate": failures / shots,
            }
            results.append(row)
            print(f"d={distance}, p={error_rate:g}, logical_error_rate={failures / shots:.6g}")
    return results

def save_csv(results, path):
    path.parent.mkdir(parents=True, exist_ok=True)
    fields = list(results[0])
    with path.open("w", newline="") as handle:
        writer = csv.DictWriter(handle, fieldnames=fields)
        writer.writeheader()
        writer.writerows(results)

def save_plot(results, path):
    import matplotlib.pyplot as plt

    path.parent.mkdir(parents=True, exist_ok=True)
    for distance in sorted({row["distance"] for row in results}):
        rows = [row for row in results if row["distance"] == distance]
        plt.loglog(
            [row["error_rate"] for row in rows],
            [max(row["logical_error_rate"], 0.5 / row["shots"]) for row in rows],
            marker="o", label=f"d={distance}",
        )
    plt.xlabel("Physical error probability p")
    plt.ylabel("Logical error rate")
```

```

plt.title("IID-noise surface-code baseline with MWPM")
plt.grid(True, which="both", linestyle=":", alpha=0.5)
plt.legend()
plt.tight_layout()
plt.savefig(path, dpi=180)
plt.close()

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--shots", type=int, default=10_000)
    parser.add_argument("--output-dir", type=Path, default=Path("results"))
    args = parser.parse_args()
    results = run_baseline(shots=args.shots)
    save_csv(results, args.output_dir / "iid_baseline.csv")
    save_plot(results, args.output_dir / "iid_baseline.png")
    print(f"Saved results to {args.output_dir / 'iid_baseline.csv'}")
    print(f"Saved plot to {args.output_dir / 'iid_baseline.png'}")

if __name__ == "__main__":
    main()

```

How it connects: Coordinates the full Monte Carlo workflow, calculates logical failures, and writes the CSV table and log-log plot.

14. tests/test_mwpm_setup.py

Original setup test. Complete source code and explanation of its role in the project.

Complete file contents

```
from src.circuits.surface_code import create_surface_code
from src.decoders.mwpm import create_mwpm_decoder

circuit = create_surface_code(
    distance=3,
    rounds=3,
    error_rate=0.001
)

decoder = create_mwpm_decoder(circuit)

print("Circuit created successfully")
print("MWPM decoder created successfully")
print(decoder)
```

How it connects: Checks the original circuit-to-decoder integration.

15. tests/test_iid_baseline.py

IID baseline tests. Complete source code and explanation of its role in the project.

Complete file contents

```
import pytest

from experiments.baseline_mwpm import run_baseline
from src.circuits.surface_code import create_surface_code
from src.noise.iid_noise import get_iid_noise


def test_iid_noise_rejects_invalid_probability():
    with pytest.raises(ValueError):
        get_iid_noise(-0.1)
    with pytest.raises(ValueError):
        get_iid_noise(1.1)


def test_surface_code_rejects_invalid_parameters():
    with pytest.raises(ValueError):
        create_surface_code(4, 4)
    with pytest.raises(ValueError):
        create_surface_code(3, 0)


def test_zero_noise_has_no_logical_failures():
    result = run_baseline(distances=(3,), error_rates=(0.0,), shots=100)
    assert result[0]["logical_failures"] == 0


def test_baseline_sweeps_requested_points():
    result = run_baseline(distances=(3, 5), error_rates=(0.001, 0.003), shots=10)
    assert len(result) == 4
    assert all(0.0 <= row["logical_error_rate"] <= 1.0 for row in result)
```

How it connects: Protects the baseline contract by checking invalid inputs, zero-noise behavior, and valid sweep results.

Final section: running and interpreting the baseline

Run these commands from the repository root:

```
PYTHONPATH=. pytest -q  
MPLCONFIGDIR=/tmp/qc-matplotlib PYTHONPATH=. python experiments/baseline_mwpm.py
```

The experiment writes `results/iid_baseline.csv` and `results/iid_baseline.png`. The CSV contains distance, physical error rate, rounds, shots, logical failures, and logical error rate. The plot uses logarithmic axes and one curve per code distance. Larger distances should generally reduce logical errors at sufficiently low physical error rates.

Next research stage

After the IID curves are trusted, implement `src/noise/ou_noise.py` using Euler-Maruyama discretization, verify $C(\delta t) = \exp(-\delta t / \tau_c)$, and then compare standard MWPM with a decoder that incorporates temporal history T .